

PS-02

Log File Anomaly Explainer Tool

AI-Powered CLI for On-Call Engineers — Instant Root Cause Analysis from Log Files

AI Engine	Google Gemini LLM
Language	Python 3.10+
Architecture	Tool-Using Agent (4 tools, up to 10 iterations)
Team	Team Apex Intelligence · 2026

1. Problem Statement

On-call engineers are overwhelmed by hundreds or thousands of log lines during a live outage. Three core challenges exist:

- Information Overload: No quick mechanism to isolate the critical failure block from thousands of lines.
- Slow Root Cause Analysis: Manually scrolling logs wastes critical minutes during live incidents.
- No Actionable Guidance: Engineers must mentally translate cryptic log messages into remediation steps under pressure.

“On-call engineer pastes 100s of log lines and asks: ‘What went wrong?’ — No automated, intelligent tool exists that takes raw log input and returns structured, actionable RCA.”

2. Our Solution

A two-mode Python CLI tool: fast Direct LLM explain and an agentic investigate mode with iterative LLM tool calls.

Mode	Description
explain (Direct)	Scans log → finds highest-severity error → extracts ±20-line context → sends to Gemini → outputs Markdown RCA report in seconds.
investigate (Agent)	LLM autonomously calls 4 tools iteratively (up to 10 rounds) to investigate the full log before producing a deep analysis.
Output Formats	Rich console (colour terminal UI) · Markdown report file · JSON (for pipeline integration)

3. System Architecture

Five tightly-integrated Python modules plus the Google Gemini LLM API:

Component	Module	Responsibility
CLI Entry	cli.py	Click commands: explain / scan / investigate. Routes output to console, JSON, or Markdown.
Anomaly Parser	parser.py	Regex severity scan (scores 1–6). Identifies primary error line, extracts ±20-line context window.
Data Model	AnomalyBlock	Dataclass: primary line #, context lines, severity score, file path, total lines, context range.

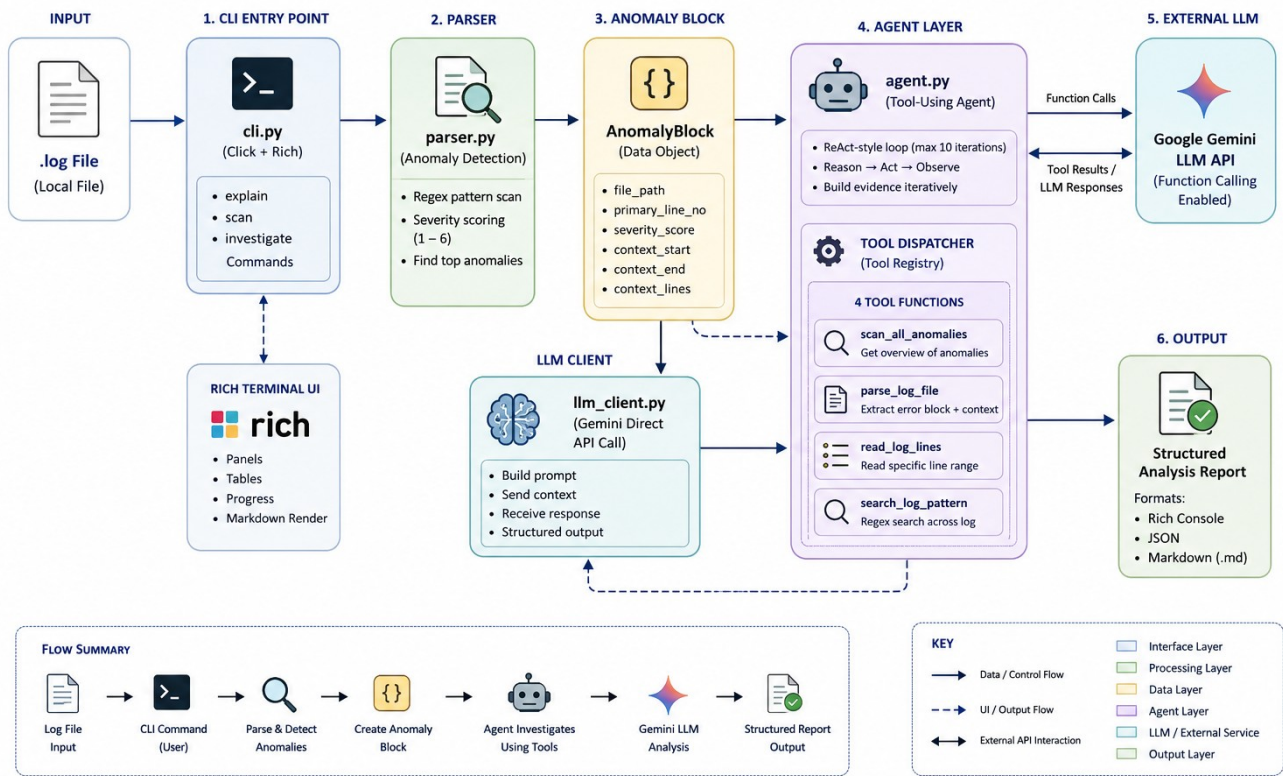
LLM Client	llm_client.py	Constructs SRE-persona prompt, calls Gemini API, parses structured response sections.
Agent Engine	agent.py	Tool-using loop: dispatches LLM function calls, collects results, iterates up to 10 times.
Tool Registry	tools.py	4 functions: scan_all_anomalies, parse_log_file, read_log_lines, search_log_pattern.

Data flow:

```
.log File → cli.py → parser.py → AnomalyBlock
├── llm_client.py (Direct mode)
├── agent.py + tools.py (Agent mode)
└── Google Gemini LLM API → Markdown Report
```

PS-02 Log File Anomaly Explainer Tool – System Architecture

AI-Powered CLI for On-Call Engineers — Instant Root Cause Analysis from Log Files



4. Technology Stack

Technology	Role & Reason
Python 3.10+	Core language — dataclasses, type hints, pathlib, re (regex engine).
Google Gemini	LLM analysis + function calling via google-genai SDK.
Click	CLI framework — clean sub-command routing (explain / scan / investigate).
Rich	Terminal UI — colour panels, tables, live progress, inline Markdown rendering.
python-dotenv	Secure API key loading from .env file.
uv / pytest	Fast package manager and unit test framework.

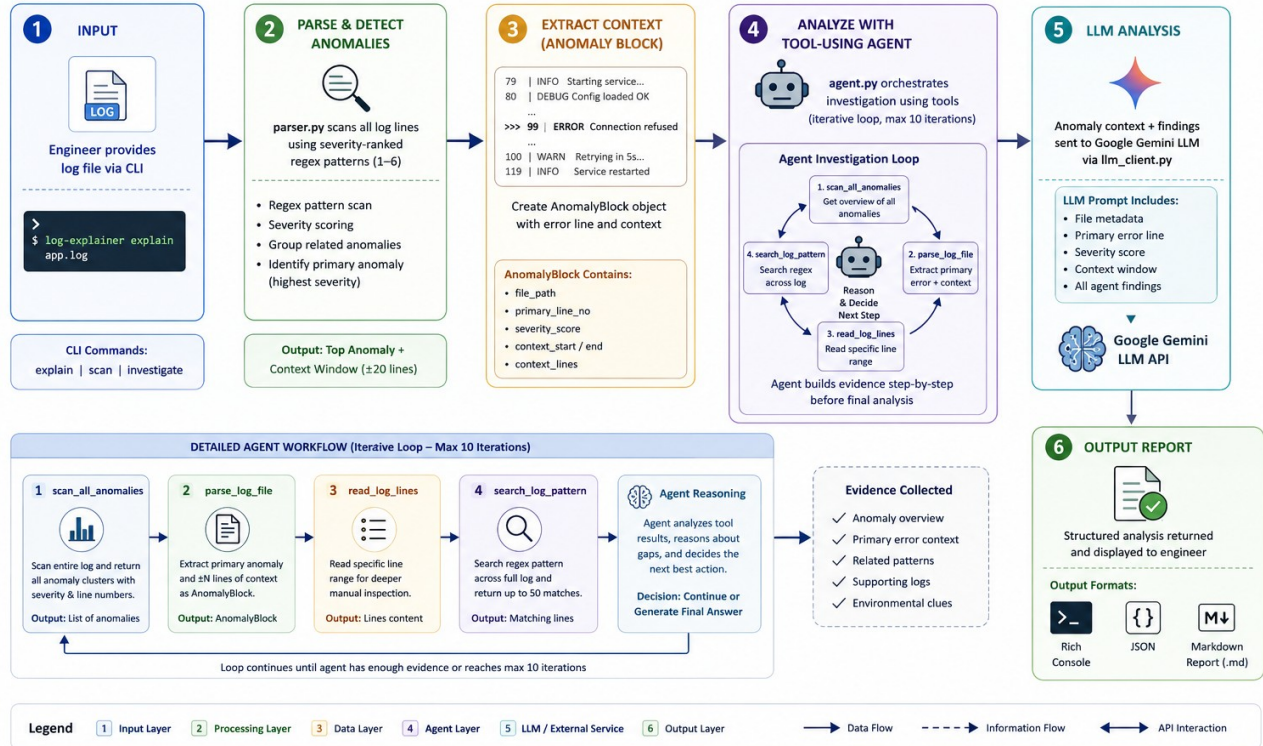
5. Workflow

5.1 Direct Explain (explain command)

1. Input: Engineer runs: `log-explainer explain app.log`
2. Parse & Score: `parser.py` scans every line against severity patterns (FATAL=6 ... WARNING=1).
3. Extract Context: ± 20 lines around the primary error extracted. Primary line marked with `>>>`.
4. LLM Analysis: Structured SRE-persona prompt submitted to Gemini with full context block.
5. Report Output: Sections parsed — Root Cause / Probable Cause / Remediation Steps / Confidence (HIGH|MEDIUM|LOW).

PS-02 Log File Anomaly Explainer Tool — Workflow

End-to-End Flow: From Log File Input to Structured Root Cause Analysis Report



```
79 | INFO Starting service... 80 | DEBUG Config loaded OK
81 | WARN Disk usage at 87% ... >>>99 | ERROR Connection refused
100 | WARN Retrying in 5s... 119 | INFO Service restarted
```

5.2 Agent Investigation (investigate command)

6. agent.py initialises Gemini with all 4 tool definitions as function declarations.
7. LLM decides which tools to call — typically starts with `scan_all_anomalies` for an anomaly map.
8. Tool results are appended to conversation history; LLM reasons over them and calls more tools.
9. Loop repeats up to 10 iterations with live Rich UI progress. Final structured analysis produced.

6. Anomaly Detection Engine

6.1 Severity Scoring

Pattern	Examples	Score
FATAL / CRITICAL / panic	FATAL: unhandled exception, kernel panic	6
OOM / segfault / killed	Out of memory: Kill process, segfault	6 / 5
EXCEPTION / Traceback	NullPointerException, Traceback	4
ERROR	ERROR connecting to DB, ERROR 500	3
Timeout / Conn refused	operation timed out, Connection refused	2
HTTP 4xx/5xx	HTTP 503 Service Unavailable	2
WARNING / WARN	WARN: high memory, disk at 87%	1

6.2 Prompt Engineering

System instruction sets SRE-Expert persona with strict ## section headers for deterministic parsing. User prompt injects: file path, total lines, primary line number, context range, severity score, and the formatted context block. Output parsed by section header search — no regex on the response side.

7. Sample Analysis — Thunderbird_2k.log

Field	Value
Primary Line	#99
Context Range	Lines 49–149
Severity	2 / 6
Confidence	HIGH
Model	Gemini Flash
Anomalies Found	4 across 2,000 lines

Root Cause

The sendmail service fails to deliver mail because no process is listening on port 25 of 127.0.0.1 — the local MTA is not running or misconfigured.

Probable Cause

MTA crash across nodes (eadmin1, badmin1) correlated with gmetad timeouts, indicating broader cluster instability — possibly OOM-induced process termination.

Remediation Steps

- 10.systemctl status sendmail — verify service state
- 11.netstat -plnt | grep :25 — confirm nothing listening on port 25
- 12.Verify FQDN in /etc/hosts · Check OOM events in dmesg · Check disk saturation

Additional Anomalies

- vesafb probe error — GPU framebuffer init failure (non-critical)
- sshd port conflict — SSH bind failure on secondary interface
- Cluster instability — gmetad monitoring timeouts across nodes

8. Applications & Real-World Use Cases

Use Case	User	Benefit
Incident Response	On-call SRE / DevOps	Instant AI-powered RCA during live outages — no manual log triage.
Post-Mortem Analysis	Engineering Manager	Auto-generate root-cause reports from historical log archives.
CI/CD Pipeline QA	Build Engineers	Detect failure patterns in test run logs; fail build with AI explanation.
DevOps Automation	Platform Engineering	Trigger alerts and automated remediation on detected anomaly patterns.
Developer Onboarding	Junior Engineers	New team members understand production errors faster via AI context.
Multi-Service Monitoring	SRE / Platform Teams	Batch-scan logs from microservices, cloud infra, and servers in one run.

9. Key Takeaways

Feature	Implementation	Impact
Automated Detection	Severity-ranked regex, 7 pattern tiers	Finds anomalies instantly across 10+ error types — zero ML required
Context-Aware AI	±20-line window with >>> primary marker	Gives Gemini the system state needed for accurate RCA
Agent Architecture	4-tool iterative loop, up to 10 iterations	LLM investigates complex multi-anomaly logs like an expert SRE
Production Ready	Rich CLI, JSON/Markdown output, unit tested	Deployable in real on-call workflows and CI/CD pipelines
Zero ML Dependency	Pure regex + LLM API	No training, no dataset, no GPU — runs anywhere Python 3.10 runs

```
log-explainer explain app.log → Instant AI-Powered Root Cause Report
```

Team PS-02 · Log File Anomaly Explainer · 2026